

Vinay Kumar Erla
Sr. Full Stack Java Developer
[AWS Certified Developer](#)
Email: vinayke959@outlook.com
Mobile: 3463331675



SUMMARY

Results-driven Full Stack Developer with 7 years of experience designing and developing scalable, high-performance web applications. Expertise in building dynamic, responsive user interfaces with React and Angular, complemented by robust back-end development using Spring Boot, Spring Web Flux, and relational databases. Proven ability to deliver end-to-end solutions by integrating cloud services and modern frameworks. Adept in Agile (SCRUM) and Waterfall methodologies, ensuring timely and high-quality deliverables.

TECHNICAL SKILLS

Category	Details
Programming Languages	Java, JavaScript, TypeScript, Python, SQL, and Shell Scripting.
Web Technologies	HTML5, CSS3, RESTful APIs, WebSockets, JSON, AJAX, and Microservices Architecture.
Front-end Technologies	React JS, Angular, Next.js, Node.js, Bootstrap, JavaScript, TypeScript, CSS, and HTML.
Spring Framework	Spring Boot, Spring Web Flux, Spring MVC, Spring Security, Spring Data JPA
Testing and Quality Assurance	Jest, JUnit, and Mockito using Test-Driven Development (TDD) and Integration testing
Database Proficiency	Oracle, SQL Server, MySQL, PostgreSQL, and NoSQL databases like Cosmos DB and MongoDB
Cloud and Kubernetes Expertise	AWS (EC2, S3, Lambda, RDS, EKS), Azure Kubernetes Service (AKS), Elastic Kubernetes Service (EKS), Terraform
Version Control and CI/CD	Git, GitHub, Bitbucket, Jira, Jenkins, Azure DevOps, Docker, Kubernetes

PROFESSIONAL EXPERIENCE

Client: Sherwin Williams, Cleveland, Ohio
Role: Senior Full Stack Developer

July 2023 – present

Responsibilities:

- Led the design and implementation of microservices using **Spring WebFlux**, achieving an improvement in system responsiveness and reduction in server resource consumption by leveraging non-blocking reactive streams. Enabled the application to handle many concurrent users and data streams efficiently, thereby enhancing scalability and responsiveness.
- Designed and developed high-performance, scalable user interfaces using React.js, leveraging advanced concepts like Hooks, Context API, custom hooks, and implementing lazy loading with code-splitting to optimize performance and user experience.
- Implemented **Redux Toolkit** to manage complex application states and integrated **Redux-Saga** for handling asynchronous operations like API calls to microservices.
- Built reusable, modular, and accessible UI components with **Material-UI** and custom styling using **CSS-in-JS** approaches like styled-components.
- Enhanced user experience with **client-side caching** and offline functionality using **Service Workers** for a PWA (Progressive Web App) approach.

- Implemented **Domain-Driven-Design** principles like **Domain Factory** Design Pattern to design the microservices architecture, ensuring that each service is modeled around a business domain.
- Integrated Domain microservices with Oracle SaaS instances using **WebClient** for asynchronous, non-blocking communication with **OracleSCM** REST endpoints, facilitating efficient data exchange and process orchestration.
- Collaborated with cross-functional teams to define the API specifications, ensuring alignment with business requirements, and facilitating seamless integration between micro services and oracle SaaS.
- Developed RESTful APIs in Backend-For-Frontend Microservice to aggregate and expose functionalities provided by Domain Microservices serving as a unified gateway for client applications.
- Employed WebClient in all microservices for external API calls, leveraging its reactive features to handle multiple concurrent requests efficiently without compromising system performance.
- Leverage the reactive programming model of Spring **WebFlux** to handle asynchronous streams of data and operations, enabling the services to be more scalable and efficient under high loads.
- Implemented non-blocking request handling in service and repository layers, ensuring that the entire stack is reactive and efficient.
- Implemented **backpressure strategies** in **Spring WebFlux** using **Project Reactor**, optimizing **Mono and Flux** pipelines with **onBackpressureBuffer()**, **onBackpressureDrop()**, and **onBackpressureLatest()** to efficiently manage high-throughput data streams and prevent memory overload.
- Implemented **OAuth2** and **JWT** for secure microservices communication, enhancing system security and data protection. Utilized **Spring Security** to enforce authentication and authorization policies across the microservices, ensuring secure access to the application's resources and protecting sensitive data.
- Implemented **Event-Driven Messaging** by Integrating Azure Service Bus to process asynchronous messages between microservices.
- Configured **OpenTelemetry** exporters to send telemetry data to external observability platforms (e.g., Prometheus, Grafana, Dynatrace), facilitating comprehensive system monitoring and alerting capabilities.
- Implemented comprehensive error handling and resilience patterns within microservices to manage exceptions and maintain stability during API communication failures or service downtimes using spring webflux through Global Error Handling annotations like **"@ControllerAdvice"**.
- Use testing frameworks like **JUnit**, **Mockito**, and **SpringTest** to write unit tests and integration tests for your microservices to ensure they behave as expected.
- Configured Azure Kubernetes Service (**AKS**) for the deployment of microservices, optimizing resource utilization and enabling automatic scaling to accommodate varying loads.
- Leveraged the terraform scripts to create Azure Resources like Cosmos DB and Azure Service Bus. Integrated Cosmos DB and Azure Service Bus Topics and Queues into Microservice.
- Leveraged Stop-light tool to create API contracts and invoked them into Microservice via Open API Generator plugin
- Exposed microservices through Azure API Management (**APIM**) gateway, configuring policies for rate limiting, CORS, and API key validation to manage access and usage of the exposed APIs.
- Integrated the exposed APIs with **Power Apps custom connectors**, enabling the development of custom business applications that interact with the microservices backend, thereby extending the application's reach and usability within the organization.
- **Environment:** Java 1.17, React Js, Spring Boot, Spring Web Flux, Web Client, AKS, APIM Gate Way, Dynatrace and open Telemetry, Agile and Water Fall Model

Responsibilities:

- Creating a **single page** web application with **Angular** as the front end, **Springboot** as the back end, and **PostgreSQL** as the database, several Spring Boot Rest APIs running under Java 8 were developed.
- Implemented the **Bootstrap** and **Angular material** together to create a more appealing and functional user interface.
- Creation of **components** to modularize and organize the UI of the Angular application and also Implemented the data binding and event binding using components.
- Creation of **Templates** to define the structure and HTML content of a component and in creating a component's visual representation and determining how it should display the data and Bind data to templates using **property** and **event bindings**.
- **Implemented routing** to handle the **navigation** from one view to the next in the application and **built-in directives** to **manage forms, lists, styles** in the application.
- Implemented advanced techniques such as host binding, host listener, and dependency injection to create complex attribute directives. Employed template references to dynamically manipulate the contents of ng-template elements and develop adaptable templates.
- Designed **non-blocking microservices** with **Reactive Streams**, utilizing **Reactive Backpressure Strategies** in **Kafka** and **RabbitMQ consumers** to handle dynamic request loads, ensure controlled data processing, and prevent resource starvation in event-driven architectures.
- Implemented the Basic Authentication, **Token-based Authentication**, **role-based authorization**, **CSRF protection** and **Session management** using **spring security**.
- Implemented **reactive REST APIs** using **Spring WebFlux** for **non-blocking data retrieval and manipulation**, enabling clients to interact with the server efficiently via **asynchronous HTTP requests and streaming response**.
- Developed Kafka message producers using Kafka Template to send messages to a Kafka topic. Implemented message processing logic such as transforming messages, performing business logic on messages, and persisting messages to the database.
- Designed and implemented complex database schemas using PostgreSQL and integrated them with Spring Boot applications. Built Spring Data JPA repositories for PostgreSQL databases and implemented CRUD operations for multiple entity classes.
- Developed a stand-alone microservice using Node.js in Express.js environment and integrated with MongoDB.
- Enhanced non-blocking I/O capabilities in Node.js, including the development of libraries that simplify event handling and asynchronous programming for the developer community.
- Spearheaded the integration of Apache Kafka with Node.js services, leveraging Amazon Managed Streaming for Kafka (MSK) to establish a robust data streaming and processing pipeline, enhancing the application's ability to handle real-time data streams efficiently.
- Performed **Distributed tracing**, utilizing the complete view of the request-response flow, to quickly identify and resolve issues in the complex microservice-based architectures.
- Created a separate test environment for Spring Boot back-end, including all necessary dependencies and configurations and used **JUnit** to write the tests that cover the various functionality of the back-end, such as testing APIs, services, and databases.
- Build servers using **AWS** which includes importing necessary volumes, launching the **EC2 instance**, **IAM**, **ELB**, **Route 53** and **SNS as per the architecture**. Deployed the web application using **Amazon EC2** instance and configure the web server **Nginx** to serve the application

- Constructed **S3** lifecycle of Applications & Database logs, which includes deleting old logs, archiving logs based on the retention policy and granting rights to authorized users and used **AWS S3** for database backup regularly and save snapshots of data.
- Secured the web application by setting up firewalls, SSL certificates, and security groups, to restrict access to the EC2 instance and protect sensitive data. Developed a secure document exchange portal using Spring Boot and AWS S3, with infrastructure managed through **Terraform**. Implemented encryption and fine-grained access control, significantly enhancing data security and user trust.
- implemented a robust monitoring framework using AWS **CloudWatch** for a containerized microservices architecture, configured AWS CloudWatch for real-time monitoring of application logs, metrics, and events to ensure high availability and performance.
- Implemented **Docker** and **Kubernetes** for the deployment, scaling, and management of containers, utilizing features such as orchestration, image creation, and configuration management, showcasing expertise in their usage.
- Integrated **DataDog** with web application to monitor infrastructure performance, Trace requests, Log management, Alerting and custom metrics.

Environment: Java 1.17, Angular, Maven, Spring Webflux, Kafka, XML, Agile, PostgreSQL, Distributed tracing, Junit, Git, Amazon Web Services, Datadog, Docker

Client: First Solar Inc, Tempe, AZ
Role: Full Stack Developer

September 2021 – March 2022

Responsibilities:

- Contributed to a single page application and involved in various phases of Software Development Life Cycle (**SDLC**) in agile environment like **requirement** gathering, **Design**, **Analysis** and Code development and creating **usecase** diagrams, **class** diagrams, and **sequence** diagrams, which help to define and visualize the relationships between the different components of the system.
- Developed the UI of the application using **React.js** and breaking down UI into smaller react components which are reusable across the application using **React's modular architecture**.
- Implemented routing using **React Router** and navigate between pages in a single-page application (SPA) and restricted access to certain routes and components based on user roles and permissions by utilizing react guards.
- Leveraged **React's Virtual DOM** to optimize the UI updates and improve performance in the current project. Extensively used **React Hooks** to add state and other features to functional components for simpler, maintainable code.
- Developed over RESTful APIs with **Express.js** and integrated with **Amazon API Gateway** for secure and scalable access. Ensured API security through the implementation of JSON Web Tokens (JWTs) and bcrypt for authentication and authorization and custom middleware for error handling and data validation. Utilized Mongoose, an ODM library for Node.js, to work with MongoDB and execute MongoDB driver for defining document and collection structures.
- Implemented event-driven, **non-blocking I/O model**, enabling the development of efficient and scalable web applications. Experienced in leveraging the Node.js module system to structure reusable and maintainable code across various projects.
- Utilized asynchronous programming using callbacks, promises, and async/await syntax to optimize web application performance and user experience and utilizing **npm** (Node Package Manager) for managing project dependencies and publishing reusable packages.
- Implemented an **event-driven architecture** with **AWS Lambda** and **SNS/SQS**, enabling efficient message processing between microservices, which enhanced application responsiveness and scalability.
- Integrating **AWS Lambda** with other AWS services such as Amazon API Gateway, S3, DynamoDB, and RDS for full-stack application development.

- Utilized AWS **EKS** for deploying microservices that consume and process data streams with **Kafka**, providing insights in real-time to end-users.
- Integrated **Amazon CloudWatch** with **AWS EKS** for monitoring, logging, and tracing of Node.js microservices, enabling proactive issue detection and resolution, and improving application reliability.
- Integrated Terraform with Jenkins and GitHub Actions to automate the provisioning of AWS infrastructure as part of the CI/CD pipeline, enabling seamless, automated deployments of microservices with each code commit.
- Implemented **Black Duck** and **SonarQube** together, to get a comprehensive view of the security and quality of the project. Utilized Black Duck to identify and manage open-source components and used SonarQube to identify and fix code quality and security issues.
- Implemented **Jest unit testing**, leveraging its snapshot testing, parallel testing, and mocking features, and utilized TypeScript to ensure code accuracy and efficiency.

Environment: React.js, Node.js, Agile, MongoDB, Amazon Web Services, Black Duck and SonarQube

Client: CALTRANS, Sacramento, CA.

Jan 2019 – July 2021

Role: Full Stack Java Developer

Responsibilities:

- Designed and developed the application using **Agile Scrum** methodology by breaking down the project to several sprints. Designed and developed Online Application using **Spring MVC Framework** and **React.js**
- Customized Web Portal using React Material UI to change the colors, font, style and components of web application.
- Implemented React hooks, including **useState** and **useEffect**, for state management and side effects in functional components.
- Utilized connect function provided by React Redux to create a higher-order component that can read data from the store and dispatch actions to update it and implemented Centralized state management.
- Developed web application using Spring MVC, configured Spring MVC for routing, handling requests and rendering views, separated concerns with MVC design pattern, implemented security and RESTful web services with caching integration and integrated with React.js.
- Configured the DispatcherServlet to handle incoming requests and route them to the appropriate controller based on URL patterns.
- Configured the HandlerMapping to map incoming requests to the appropriate controller based on URL patterns or other criteria. Implemented custom HandlerMapping implementations to handle specific types of requests or to provide custom logic for mapping requests to controllers.
- Implemented data to a database using **Spring's DAO** (Data Access Object) layer and it's exception hierarchy to handle data access exceptions and provide meaningful error messages to the user.
- Set up the connection to the Oracle database from the Spring MVC backend and used **JDBC driver to communicate with the Oracle database** to retrieve and manipulate data.
- Implemented disaster recovery solutions using Oracle Data Guard to create a standby database that can be used to quickly recover from a disaster or system failure.
- Configured Oracle Automatic Storage Management (ASM) to manage database files and ensure data availability in case of storage failures.
- Developed and maintained RESTful APIs using Node.js and Express.js for internal and external applications, resulting in an increase in backend efficiency and data processing speed.
- Integrated **CodePipeline with AWS CodeBuild** and other build tools to compile and test the code, and then deploys the code to the target environment.
- Created and set up repositories in AWS CodeCommit to store my project's source code. This would involve creating repositories, setting up access control, and configuring repository settings.

- Designed and Created data warehouse solutions in BigQuery, optimizing schema data, design, data ingestion pipelines and Implemented data modeling techniques for querying and reporting.
- Implemented **Veracode scanning tool** to identify a wide range of security vulnerabilities in web applications, including **cross-site scripting (XSS)**, **SQL injection**, and other common security threats.
- Utilized **Veracode** to improve the overall security of the applications and reduce the risk of data breaches and other security incidents and track the progress of the security improvement initiatives.
- Using distributed tracing to monitor the performance of react.js and Spring MVC applications and identify opportunities for optimization and make data-driven decisions about how to improve performance.
- Implemented Jasmine's asynchronous testing features, such as the done function, to test asynchronous code, such as AJAX requests and also Implemented mocks to simulate functions and objects, and verify that they are called correctly during the execution of my tests.
- Implemented **App Dynamics monitoring tool** to create custom dashboards to visualize key performance metrics and insights and share them with stakeholders.
- Optimize the performance of the application by identifying bottlenecks, analyzing slow transactions, and tuning configurations.

Environment: HTML 5, CSS 3, JavaScript, Ajax, JSON, Bootstrap, jQuery, React.js, Node.js, Agile, Oracle DB, AWS Code Pipeline, Python, Veracode, Distributed Tracing, jasmine, Mockito, ServiceNow, App Dynamics Monitoring tool.

Client: Licious, Bangalore, India
Role: Full Stack Java Developer

Jan 2018 – Jan 2019

Responsibilities:

- Involved in Requirements gathering, Analysis and Design of the application. Used **OOD concepts and Core Java** in overall design and development of the backend and used HTML to structure the content and layout of a web page and make use of CSS (Cascading Style Sheets) to style and format the content of a web page.
- Creating a **single page** web application with **React** as the front end, **Springboot** as the back end, and **PostgreSQL** as the database, several Spring Boot Rest APIs running under Java 8 were developed.
- Creation of **components** to modularize and organize the UI of the React application and also Implemented the data binding and event binding using components.
- Implemented **Redux** middleware like Redux **Thunk** and Redux **Saga** for handling asynchronous actions and side effects in React application.
- Utilized **Next.js API routes** to build serverless APIs for handling user authentication and data processing, improving the security and scalability of the application.
- **Implemented routing** to handle the **navigation** from one view to the next in the application and **built-in directives to manage forms, lists, styles** in the application.
- Used the Spring MVC framework to create RESTful web services that utilize centralized request handling to provide a scalable and flexible solution for handling large volumes of requests.
- Implemented internationalization and localization features using message bundles and resource bundles to support multiple languages and locales in the application
- Implemented Oracle's transaction management and locking mechanisms to ensure data consistency and prevent data corruption.
- Configured Log4j in the Spring MVC application to send log messages to a centralized logging system, which made it easier to monitor and manage logs more efficiently.
- Implemented **Jest's features** such as **snapshot testing, mocking, and assertion functions** to ensure accurate and comprehensive testing.

- Implemented TypeScript for static type checking, which helps catch errors at development time rather than at runtime.
- **Created and configured the build pipeline** for the front-end and back-end applications in **Azure DevOps**. This includes tasks such as installing dependencies, building the applications, and running tests.
- Integrated the front-end and back-end applications with other services and tools in Azure DevOps, such as **Azure Functions, Azure Cosmos DB, and Azure Event Grid**.
- Utilized Azure DevOps to monitor the performance of the front-end and back-end applications in production. This includes tasks such as setting up alerts, analyzing logs, and monitoring resource utilization.
- Utilized Jira Service Desk to create, manage, and track issues and for collaboration with Stakeholders.

Environment: HTML 5, CSS 3, JavaScript, Ajax, JSON, Bootstrap, JQuery, React.js, maven, Spring MVC, Agile, OracleDB, Travis CI, TypeScript, Logging framework, jest, TestNG, Jira.